```
> # Independent exact check of (41) in the 17-page manuscript.
  # Integer arrays represent polynomials whose coefficients are
  divided by G.
  # The approximate coefficients are certified by a contraction on
  the
  # real truncated polynomial ring with norm
  sum(abs(coefficients)).
  VerifyFiniteLogConvexity := module()
      export Run;
      local G, K, s, radix, blockLength, radixPower, bias,
  weights,
            NormUnits, RoundUnits, Pack, Unpack, Multiply,
            ThetaAndDerivative, Inverse;

      NormUnits := proc(A, count)
          local j, result;
          result := 0;
          for j from 0 to count-1 do
              result := result+abs(A[j]);
          end do;
          return result;
      end proc;

      RoundUnits := proc(value)
          if value < 0 then
              return -iquo(-value, G);
          end if;
          return iquo(value, G);
      end proc;

      # Evaluate the signed coefficient vector at the integer
  radix.
      Pack := proc(A, first, count, depth)
          if first >= count then
              return 0;
          end if;
          if depth = 0 then
              return A[first];
          end if;
          return procname(A, first, count, depth-1)
               + radixPower[depth-1]*
                 procname(A, first+blockLength[depth-1], count,
  depth-1);
      end proc;

      Unpack := proc(encoded, A, first, count, depth)
```

```
        local high, divisor;
        if first >= count then
            return NULL;
        end if;
        if depth = 0 then
            A[first] := RoundUnits(encoded-radix/2);
            return NULL;
        end if;
        divisor := radixPower[depth-1];
        high := iquo(encoded, divisor);
        procname(encoded-high*divisor, A, first, count,
depth-1);
        procname(high, A, first+blockLength[depth-1], count,
depth-1);
        return NULL;
    end proc;

    # Truncated product followed by rounding toward zero on the
grid.
    # The bias makes each signed convolution coefficient a radix
digit.
    Multiply := proc(A, lenA, B, lenB, count)
        local aCount, bCount, depth, encoded, result;
        aCount, bCount := min(lenA, count), min(lenB, count);
        if 2*NormUnits(A, aCount)*NormUnits(B, bCount) >= radix
then
            error "the convolution coefficients exceed the
packing bound";
        end if;
        depth := 0;
        while blockLength[depth] < count do
            depth := depth+1;
        end do;
        encoded := irem(Pack(A, 0, aCount, depth)*
                        Pack(B, 0, bCount, depth)+bias[depth],
                        radixPower[depth]);
        if encoded < 0 then
            encoded := encoded+radixPower[depth];
        end if;
        result := Array(0..count-1);
        Unpack(encoded, result, 0, count, depth);
        return result;
    end proc;

    ThetaAndDerivative := proc(P, lenP, count, derivativeCount)
        local F, D, power, powerLength, k, j, shift, c;
```

```
        F := Array(0..count-1, fill=0);
        D := Array(0..derivativeCount-1, fill=0);
        F[0], D[0] := G, -G;
        for j from 0 to min(lenP, count)-1 do
            F[j] := F[j]-P[j];
        end do;
        power, powerLength := P, lenP;
        for k from 2 to K do
            shift := k*(k-1)/2;
            if shift >= count then
                break;
            end if;
            c := weights[k];
            for j from 0 to min(powerLength,
derivativeCount-shift)-1 do
                D[j+shift] :=
D[j+shift]+RoundUnits(k*c*power[j]);
            end do;
            power := Multiply(power, powerLength, P, lenP,
count-shift);
            powerLength := count-shift;
            for j from 0 to powerLength-1 do
                F[j+shift] := F[j+shift]+RoundUnits(c*power[j]);
            end do;
        end do;
        return [F, D];
    end proc;

    Inverse := proc(D, count)
        local V, m, nextLength, X, j;
        if D[0] <> -G then
            error "the constant term of the derivative is not
-1";
        end if;
        V := Array(0..0, fill=-G);
        m := 1;
        while m < count do
            nextLength := min(2*m, count);
            X := Multiply(D, nextLength, V, m, nextLength);
            for j from 0 to nextLength-1 do
                X[j] := -X[j];
            end do;
            X[0] := X[0]+2*G;
            V := Multiply(V, m, X, nextLength, nextLength);
            m := nextLength;
        end do;
```

```
        return V;
    end proc;

    Run := proc()
        local limit, target, radiusUnits, j, k, m, nextLength,
width,
              P, nextP, F, D, V, tail, correction, shift, ratio,
              derivativeRatio, thetaTail, derivativeTail,
evalError,
              product, normV, displacement, contraction, n, a1,
a2, a3,
              upperSquare, gap, bestIndex, bestNumerator,
bestDenominator;

        limit := 60000;
        target := limit+1;
        G := 2^192;
        radiusUnits := 2^92;
        K := 16;
        s := 1546246693/5000000000;
        radix := 2^448;

        blockLength := Array(0..16);
        radixPower := Array(0..16);
        bias := Array(0..16);
        blockLength[0], radixPower[0], bias[0] := 1, radix,
radix/2;
        for j from 1 to 16 do
            blockLength[j] := 2*blockLength[j-1];
            radixPower[j] := radixPower[j-1]^2;
            bias[j] := (1+radixPower[j-1])*bias[j-1];
        end do;
        weights := Array(2..K);
        for k from 2 to K do
            weights[k] := (-1)^k*floor(G*s^(k*(k-1)/2));
        end do;

        P := Array(0..0, fill=G);
        m := 1;
        while m < target do
            nextLength := min(2*m, target);
            width := nextLength-m;
            F, D := op(ThetaAndDerivative(P, m, nextLength,
width));
            V := Inverse(D, width);
            tail := Array(0..width-1);
```

```
            for j from 0 to width-1 do
                tail[j] := F[m+j];
            end do;
            correction := Multiply(V, width, tail, width,
width);
            nextP := Array(0..nextLength-1);
            for j from 0 to m-1 do
                nextP[j] := P[j];
            end do;
            for j from 0 to width-1 do
                nextP[m+j] := -correction[j];
            end do;
            P, m := nextP, nextLength;
            printf("Approximation through degree %d\n", m-1);
        end do;

        if target >= G or s <= 0 or s >= 1/3 or
           NormUnits(P, target)+radiusUnits >= 3*G then
            error "the hypotheses of the norm estimates fail";
        end if;
        shift := K*(K+1)/2;
        ratio := 3*s^(K+1);
        derivativeRatio := ratio*(K+2)/(K+1);
        if derivativeRatio >= 1 then
            error "the theta tail ratios are not below one";
        end if;
        thetaTail := 3^(K+1)*s^shift/(1-ratio);
        derivativeTail := (K+1)*3^K*s^shift/(1-derivativeRatio);
        if max(thetaTail, derivativeTail) >= 1/G then
            error "the theta tails exceed the allowed error";
        end if;

        # Each rounded product costs at most target/G in norm.
The
        # error in P^k is <= target*3^(k-1)/G, so the norm of
the
        # computed power is <= 2*3^k. Rounding the scalar
weights and
        # their products costs at most another
(2*k*3^k+target)/G
        # for term k. Summing these errors and the omitted tail
        # gives the following bound for both F and its
derivative.
        evalError := (4*K^2*target*3^K+1)/G;
        F, D := op(ThetaAndDerivative(P, target, target,
target));
```

```
        V := Inverse(D, target);
        if V[0] <> -G then
            error "the approximate inverse is not a unit";
        end if;
        normV := NormUnits(V, target)/G;
        product := Multiply(D, target, V, target, target);
        product[0] := product[0]-G;
        displacement := normV*(NormUnits(F,
target)/G+evalError);

        # On the ball of radius 2^(-100), the second derivative
        # has norm <= 2/3+(2/3)/(1-2/9) < 2, using s < 1/3.
        # The following bounds certify Q -> Q - V*Psi(Q, s*z).
        contraction := (NormUnits(product, target)+target)/G
                       + normV*(evalError+2*radiusUnits/G);
        if displacement >= radiusUnits/(2*G) or contraction >=
1/2 then
            error "the contraction or the ball inclusion fails";
        end if;
        printf("Displacement < 2^(-101) and contraction factor <
1/2   PASS\n");
        # V is a unit, and Psi(Q,s*z)=0 has one formal solution.
        # The contraction thus encloses T(s*z) through degree
60000.
        printf("Coefficient errors <= 2^(-100) through degree
60000   PASS\n");

        for n from 0 to limit do
            if P[n] <= radiusUnits then
                error "a coefficient enclosure is not positive
at n = %1", n;
            end if;
        end do;

        # The exact terms through q^3 follow from
1-T+q*T^2-q^3*T^3.
        a1 := 1;
        a2 := 2*a1;
        a3 := 2*a2+a1^2-1;
        if a1*a3 <> a2^2 then
            error "the exact equality at n = 2 fails";
        end if;
        printf("n = 2   a[1] = %a, a[2] = %a, a[3] = %a
equality PASS\n",
               a1, a2, a3);
```

```
        bestIndex, bestNumerator, bestDenominator := 0, 0, 1;
        for n from 1 to limit-1 do
            if n <> 2 then
                upperSquare := (P[n]+radiusUnits)^2;
                gap :=
  (P[n-1]-radiusUnits)*(P[n+1]-radiusUnits)-upperSquare;
                if 2500000000*gap <= upperSquare then
                    error "the strict bound in (41) fails at n =
  %1", n;
                end if;
                if bestIndex = 0 or gap*bestDenominator <
  bestNumerator*upperSquare then
                    bestIndex, bestNumerator, bestDenominator :=
  n, gap, upperSquare;
                end if;
            end if;
        end do;
        printf("%d strict comparisons in (41)   PASS\n",
  limit-2);
        printf("Smallest certified lower bound at n = %d is at
  least %a*10^(-24)\n",
               bestIndex, iquo(10^24*bestNumerator,
  bestDenominator));
        printf("All checks passed.\n");
    end proc;
  end module:

  VerifyFiniteLogConvexity:-Run():
Approximation through degree 1
Approximation through degree 3
Approximation through degree 7
Approximation through degree 15
Approximation through degree 31
Approximation through degree 63
Approximation through degree 127
Approximation through degree 255
Approximation through degree 511
Approximation through degree 1023
Approximation through degree 2047
Approximation through degree 4095
Approximation through degree 8191
Approximation through degree 16383
Approximation through degree 32767
Approximation through degree 60000
Displacement < 2^(-101) and contraction factor < 1/2   PASS
Coefficient errors <= 2^(-100) through degree 60000   PASS
n = 2   a[1] = 1, a[2] = 2, a[3] = 4   equality PASS
59998 strict comparisons in (41)   PASS
Smallest certified lower bound at n = 59999 is at least 416678837903839*10^(-24
)
All checks passed.
>
```